

Name or Value? Cue Conflict Reveals When Chain of Thought Overrides Lexical Priors

Anonymous ACL submission

Abstract

Language models are misled by variable names that contradict a program’s behavior, but an accuracy drop cannot show where inside the model the name interferes. Interpretability studies find that intermediate values are computed during chain of thought, yet in their clean tasks the variable name and the computed value never disagree. We introduce a cue-conflict test for multi-step arithmetic in which each problem appears with congruent, neutral, and incongruent variable names under identical arithmetic. Across $\langle\langle n\text{-models}\rangle\rangle$ models, incongruent names reduce accuracy by $\langle\langle\text{interference-direct}\rangle\rangle$ points without chain of thought and by $\langle\langle\text{interference-cot}\rangle\rangle$ points with it. Linear probes trained on neutral problems only show that the lure value remains decodable $\langle\langle\text{lure-persistence}\rangle\rangle$, and activation patching from matched neutral problems recovers $\langle\langle\text{recovery}\rangle\rangle\%$ of lure errors. These results indicate that chain of thought $\langle\langle\text{headline-verb}\rangle\rangle$ lexical interference, and they $\langle\langle\text{headline-stance}\rangle\rangle$ the view that chain-of-thought computation is faithful.

1 Introduction

Models read code and mathematics in which names carry meaning, and names that contradict the computation reduce accuracy: on real programs, removing or corrupting identifiers degrades execution prediction (Le et al., 2025), misleading comments and hints cut output prediction by a fifth (Lam et al., 2025), and adversarial renaming has collapsed comprehension accuracy to 21% in a prior study (Anonymous, 2025). An accuracy drop, however, is compatible with three different failures: a corrupted input representation, a corrupted computation, or a correct computation misread at output. These have different implications for whether a written chain of thought can be trusted, and for how the failure might be fixed.

Interpretability work on clean tasks cannot separate them either. Kudo et al. (2026) show, with linear probes and activation patching on controlled multi-step arithmetic, that the values of intermediate variables become linearly decodable only as the chain of thought is written, and that the final answer depends causally on the chain. But in their tasks the variable names are letters. The decoded value has exactly one possible source, so nothing in the design can tell “the model computed 5” from “the model read 5 off a cue,” because no cue ever disagrees.

Cue conflict resolves this. In the Stroop task (Stroop, 1935), the word RED printed in blue ink reveals whether reading overrides colour naming; in vision, shape–texture conflict images showed that apparent shape recognition in CNNs was partly texture matching (Geirhos et al., 2019). We bring the design to arithmetic reasoning. Every problem appears in three matched versions with identical arithmetic (Figure 1): neutral names (pen=1 + cup, cup=2 + 3; pen=?), congruent names, where a number word equals the variable’s true value (five=2 + 3), and incongruent names, where it does not (two=2 + 3, true value 5, lure 2). Each version is solved with and without a written chain of thought, and linear probes trained on neutral problems only read out, at every layer and reasoning step, whether the representation holds the true value or the lure.

Our main finding is that $\langle\langle\text{headline-sentence}\rangle\rangle$. Concretely, we contribute (i) a cue-conflict extension of a controlled arithmetic benchmark with matched twins and generation rules that make the lure never correct for another reason; (ii) a layer, position and reasoning-step map of lure versus true value, with and without chain of thought, across $\langle\langle n\text{-models}\rangle\rangle$ models; and (iii) causal evidence from activation patching, with two control patches, on whether the name’s influence

```

pen=1 + cup, cup=2 + 3; pen=?
pen=1 + five, five=2 + 3; pen=?
pen=1 + two, two=2 + 3; pen=?
pen=1 + two, two=2 + 3, two=5, pen=1 + two,
pen=1 + 5, pen=6
[margin sketch:  $\log p(\text{true}) - \log p(\text{lure})$  across P2–P5,
to be drawn from a real instance once probes exist]

```

Figure 1: One problem in its three matched versions (top), the chain of thought with probe positions P2–P5 (middle), and the probe margin across reasoning steps (bottom). The incongruent name two has true value 5 and lure value 2.

explains the errors.

2 Setup

Task. We reuse the generator, difficulty levels and prompt format of Kudo et al. (2026): problems are lists of assignments over single digits with + and −, all values stay in 0..9, and three fixed same-level demonstrations precede each problem. Our main level is their Level 3 (two steps, one forward reference; $v1=d \pm v2$, $v2=d \pm d$; $v1=?$), with Levels 2 and 5 as robustness checks and Level 4 for a control described below. Probes are trained on 10,000 neutral problems and everything is evaluated on 2,000 matched sets per level; the digit expressions used for probe training are held out from the test sets.

Conditions. Each arithmetic problem yields a matched set. The *letter* condition is Kudo et al.’s format and anchors our replication. *Neutral* names are common single-token words. In the *congruent* and *incongruent* conditions exactly one variable, the target, is renamed to a number word that equals or does not equal its true value. The target is either the intermediate variable or the queried one. An *irrelevant-lure* control at Level 4 gives the number-word name to a distractor outside the queried chain, separating the lexical presence of a number from a binding conflict.

Generation rules. Exactly one variable per instance carries a number word. The lure differs from every variable value and from every operand digit in the problem, so it is never correct for another reason. Neutral words are drawn from a pool of common concrete nouns that are a single token in every context of the template under each model’s tokenizer (95 of 112 candidates for the Llama 3 tokenizer; binary operators are spaced because − otherwise merges with the fol-

lowing word), so twins tokenize to identical layouts and length is not a confound. Every incongruent instance has a neutral and a congruent twin with identical arithmetic, and all comparisons are within these sets.

Regimes. In the *chain-of-thought* regime the model writes each step in Kudo et al.’s format (cup=5, pen=1 + 5, pen=6); note that the chain writes the name next to the computed value, which may help the model recover. In the *direct* regime it writes only pen=6. The contrast between regimes is the center of the paper.

Probe positions. Using the incongruent example: P1, the name token two at its definition, is expected to decode the lure trivially and is reported only as a reference, never as evidence. P2 is the last operand of the definition, the first position where the true value can be computed. P3 is the query position before any chain of thought. P4 is the chain position where the target’s value is about to be written. P5 is the final answer position. All claims about lure interference are made at P2–P5.

Probes and patching. Probes are single linear layers trained with Kudo et al.’s recipe on the neutral condition only, then evaluated on congruent and incongruent instances; training on incongruent data would let the probe learn the lure. We report accuracy for the true value, the *lure mass* (probability on the lure class), and the *margin* $\log p(\text{true}) - \log p(\text{lure})$; the *crossover* is the first step at which the margin becomes and stays positive. Selectivity against a control task (Hewitt and Liang, 2019) accompanies every probe, over three seeds. For causation we patch residual-stream activations at the misleading name’s token positions from the matched neutral twin into the incongruent run (Zhang and Nanda, 2024), sweep layers, and measure the *recovery rate*, the share of lure errors that become correct. A patch from a different neutral word into the neutral run measures patching damage; a patch from a different lure tests whether the answer follows the new name.

Models. $\langle\langle\text{model-list}\rangle\rangle$, base models of 3B and 8B parameters from Kudo et al.’s panel, plus $\langle\langle\text{extra-models}\rangle\rangle$. Behavioural metrics are accuracy, lure rate (answers equal to the lure), *interference* (neutral minus incongruent accuracy) and *facilitation* (congruent minus neutral), with cluster-bootstrap confidence intervals over matched sets.

Model	Regime	Target	Neutral	Congr.	Incongr.	Lure rate	Interf.	Facil.
-------	--------	--------	---------	--------	----------	-----------	---------	--------

Table 1: Accuracy (%), lure rate, interference and facilitation per model, regime and target position, Level 3, 95% cluster-bootstrap intervals over matched sets.

[Figure 2: margin heatmap, generated by scripts/52_figs.py]

Figure 2: Probe margin $\log p(\text{true}) - \log p(\text{lure})$ by layer and position, incongruent condition, chain of thought (left) and direct answer (right). Positive means the true value dominates.

3 Results

Replication. On the letter condition our probes reproduce Kudo et al.’s result for `<<replication-model>>`: the queried variable becomes decodable at equation `<<replication-tstar>>` of the chain and not before (their Table 3: 5).

RQ1: behaviour. Table 1 gives accuracy, lure rate, interference and facilitation per model and regime. Incongruent names reduce accuracy by `<<interference-direct>>` points in the direct regime and `<<interference-cot>>` points with chain of thought; `<<lure-share>>` of the incongruent errors are the lure itself. The irrelevant-lure control shows `<<irrelevant-effect>>`, so `<<binding-reading>>`.

RQ2: representation. Figure 2 maps the probe margin over layers and positions P2–P5 in the incongruent condition, chain of thought beside direct answer. At P2 the lure `<<p2-reading>>`; at P3 `<<p3-reading>>`. With chain of thought the margin crosses over at `<<crossover-step>>` and `<<crossover-persistence>>`; without it `<<direct-reading>>`. At the instance level, the margin at `<<link-position>>` predicts lure errors (`<<link-coefficient>>`), placing the outcome in the `<<outcome-cell>>` cell of the outcome table.

RQ3: causation. Figure 3 plots recovery by patched layer. Patching the name positions from the neutral twin recovers `<<recovery>>`% of lure errors at `<<recovery-layers>>`, while the control patch changes `<<control-damage>>`% of correct neutral answers and the alternative-lure patch moves the answer to the new lure in `<<follow-new-lure>>`% of cases. `<<patching-reading>>`.

[Figure 3: patching recovery by layer with controls, generated by scripts/52_figs.py]

Figure 3: Recovery rate of lure errors by patched layer, with the neutral-word control patch and the alternative-lure patch.

4 Related Work

Kudo et al. (2026) established when sub-answers are computed during chain of thought on clean arithmetic; we add cue conflict to ask what that computation is sensitive to, and reuse their code and format. Faithfulness of chain of thought has been measured from the outside by perturbing prompts or chains (Turpin et al., 2023; Lanham et al., 2023; Chen et al., 2025); we measure it inside the model under a controlled conflicting cue. Behavioural work on code shows that misleading identifiers and comments hurt (Le et al., 2025; Lam et al., 2025; Anonymous, 2025); we trade realism for localisation. Work on binding asks how a value attaches to a name (Feng and Steinhardt, 2024; Wu et al., 2025); we ask whether the attachment survives a name that carries a competing value. Shi et al. (2023) add irrelevant context to arithmetic; here the distractor is the variable’s own name. The design itself is the Stroop task (Stroop, 1935) and the shape–texture conflict of Geirhos et al. (2019).

5 Conclusion

`<<conclusion-finding>>`. For faithfulness this means `<<conclusion-implication>>`: a written chain of thought `<<conclusion-verb>>` the model’s internal reliance on a misleading name, and the place where the lure survives, or fails to, is where a fix would have to act.

Limitations

The task is synthetic arithmetic with short chains, so the results need not transfer directly to real code, which is the motivation and not the finding. Models are at most 8B parameters; within-family comparisons give the direction of the trend only. Lures are hand-picked English number words; digit-bearing names such as x2, other languages and optimised lures are future work. Linear probes can miss nonlinearly encoded information; the patching experiment partly addresses this, but hidden states carry mixed information and patching cannot rule out side effects entirely, which is why two

control patches are reported. The chain-of-thought format writes the name beside the computed value at every step, which may itself aid recovery; the direct regime bounds that effect from the other side.

References

Anonymous. 2025. [prior renaming comprehension study; citation to be supplied by the lead, cited in the third person].

Yanda Chen et al. 2025. Reasoning models don’t always say what they think. *arXiv preprint arXiv:2505.05410*.

Jiahai Feng and Jacob Steinhardt. 2024. How do language models bind entities in context? In *International Conference on Learning Representations*.

Robert Geirhos, Patricia Rubisch, Claudio Michaelis, Matthias Bethge, Felix A. Wichmann, and Wieland Brendel. 2019. ImageNet-trained CNNs are biased towards texture; increasing shape bias improves accuracy and robustness. In *International Conference on Learning Representations*.

John Hewitt and Percy Liang. 2019. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*.

Keito Kudo, Yoichi Aoki, Tatsuki Kuribayashi, Shusaku Sone, Masaya Taniguchi, Ana Brassard, Keisuke Sakaguchi, and Kentaro Inui. 2026. LLMs faithfully and iteratively compute answers during CoT: A systematic analysis with multi-step arithmetics. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 1114–1153. Code: <https://github.com/keitokudo/faithful-cot-multistep-arithmetic>.

Man Ho Lam et al. 2025. CodeCrash: Exposing LLM fragility to misleading natural language in code reasoning. In *Advances in Neural Information Processing Systems*. ArXiv:2504.14119.

Tamera Lanham et al. 2023. Measuring faithfulness in chain-of-thought reasoning. *arXiv preprint arXiv:2307.13702*.

Cuong Chi Le, Minh V. T. Pham, Cuong Duc Van, Hoang N. Phan, Huy N. Phan, and Tien N. Nguyen. 2025. When names disappear: Revealing what LLMs actually understand about code. *arXiv preprint arXiv:2510.03178*.

Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H. Chi, Nathanael Schärli, and Denny Zhou. 2023. Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning*.

J. Ridley Stroop. 1935. Studies of interference in serial verbal reactions. *Journal of Experimental Psychology*, 18(6):643–662.

Miles Turpin, Julian Michael, Ethan Perez, and Samuel Bowman. 2023. Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems*, volume 36, pages 74952–74965.

Yiwei Wu, Atticus Geiger, and Raphaël Millière. 2025. How do transformers learn variable binding in symbolic programs? In *Proceedings of the 42nd International Conference on Machine Learning*.

Fred Zhang and Neel Nanda. 2024. Towards best practices of activation patching in language models: Metrics and methods. In *International Conference on Learning Representations*.

A Tokenizer checks

Per-model lists of neutral words that are one token in every context (line start, after a comma, as an operand), and the token layout of the template: `<<appendix-tokenizer>>`.

B Full layer sweeps and selectivity

`<<appendix-sweeps>>`

C Additional models and levels

`<<appendix-models>>`